

AGENT小黄书 • V0.1

Agent小黄书

从零基础到可交付的能力

完整教程导出

33 个内容单元

CONTENTS

目录

01	Agent小黄书：从零基础到变现的全链路…
	前言与准备
03	小白最短路径：30 分钟装上第一个智能体
	前言与准备
05	第 2 章 AI 编程工具进化史与 2026 格局
	第一篇 认识智能体
07	第 4 章 准备：API Key 与环境三件套
	第二篇 安装与配置
09	第 6 章 Codex 安装与上手
	第二篇 安装与配置
11	第 8 章 Ollama 本地部署
	第二篇 安装与配置
13	第 10 章 平台 Skill 与 2026 跨平台标准
	第三篇 Skill 完全指南
15	第 12 章 怎么写自己的 Skill
	第三篇 Skill 完全指南
17	第 14 章 需求与方案
	第四篇 开发流程与标准工作流
19	第 16 章 测试与上线
	第四篇 开发流程与标准工作流
21	第 18 章 智能体开发六步法（企业级）
	第四篇 开发流程与标准工作流
23	第 20 章 变现核心逻辑
	第五篇 变现指南
25	第 22 章 起步路线图与避坑
	第五篇 变现指南
27	第 24 章 法律红线与合规速查
	第六篇 安全与合规

02	导读：这本书怎么用
	前言与准备
04	第 1 章 智能体到底是什么
	第一篇 认识智能体
06	第 3 章 2026 年 8 月速览：一页看懂最新…
	第一篇 认识智能体
08	第 5 章 Claude Code 安装与上手
	第二篇 安装与配置
10	第 7 章 Hermes 与国产工具
	第二篇 安装与配置
12	第 9 章 Skill 是什么、为什么
	第三篇 Skill 完全指南
14	第 11 章 怎么找、怎么判断一个好 Skill
	第三篇 Skill 完全指南
16	第 13 章 完整示例与文件组织
	第三篇 Skill 完全指南
18	第 15 章 开发、联调与 Code Review
	第四篇 开发流程与标准工作流
20	第 17 章 救火与复盘
	第四篇 开发流程与标准工作流
22	第 19 章 标准工作流与提示词模板库
	第四篇 开发流程与标准工作流
24	第 21 章 14 条变现路径
	第五篇 变现指南
26	第 23 章 权限与安全铁律
	第六篇 安全与合规
28	附录 A：命令速查总表
	附录与资源

29

附录 B：工具与资源链接汇总（2026-08 ...

附录与资源

31

附录 D：各平台费用与模型速查表

附录与资源

33

结语

第六篇 安全与合规

30

附录 C：常见报错速查表

附录与资源

32

附录 E：小白术语表

附录与资源

01

前言与准备

Agent小黄书：从零基础到变现的全链路指南

基础版本 · V0.1

V0.1 | 按照系统化学习路径组织：分篇分章、每章有结论速览、类比表格、命令速查、安全红线、可复制模板

适用对象：零基础小白、上班族、宝妈、学生、自由职业者、想入门 AI Agent 的每一个人

导读：这本书怎么用

基础版本 · V0.1

这是一本把"智能体"讲透的**小黄书**。它围绕零基础上手、技能构建、开发 workflow、变现与安全合规，按章节提供结论速览、类比表格、命令速查和可直接复制的提示词模板，并持续维护实践内容。

三种读法，按需选择：

你的目标	读法
纯小白，想装上第一个智能体	先读「小白最短路径」→再第 1 篇→第 2 篇，按顺序照着做
会用一点，想效率翻倍	直接跳到第 3 篇（Skill 指南）+ 第 4 篇（标准 workflow 与模板库）
想靠 AI 赚钱	第 5 篇（变现）+ 第 6 篇（安全合规），第 1、2 篇快速过一遍

本书涵盖的核心方向：

- 智能体安装流程及基础教程
- 智能体 Skill 指南
- 零基础写 Skill
- 开发流程规范
- AI 智能体变现指南

重要提醒：本教程不构成任何投资、法律或医疗建议。所有工具、渠道、价格、额度政策均可能随时变化，请以各平台官网最新公告为准。教程中提及的"绕封锁/接码/第三方中转"等内容仅作信息说明，请遵守所在地法律法规，优先使用官方正规渠道。

小白最短路径：30 分钟装上第一个智能体

基础版本 · V0.1

先别通读全书再动手。 按下面 5 步走，30 分钟内你就能让第一个智能体跑起来；其余章节当字典，用到再查。

步骤	做什么	去哪看	怎么算成功
1	申请一个 API KEY（建议先选国内有免费额度的：豆包 / 智谱 / 小米）	第 4 章 第一步	在平台"API Key"页面能看到你申请的 Key
2	安装 Git / Node.js / Python 三件套	第 4 章 第二~四步	三条验证命令都通过（记得用新窗口验证）
3	装一个智能体（国内推荐 Deveco Code 或 ZCode，免魔法）	第 7 章	输入启动命令后，智能体能跟你打招呼
4	给它第一个任务："请做一个简单网页：黑色背景，中间白色文字显示 Hello"	第 6 章 App 上手	双击生成的 HTML 文件，浏览器里能看到页面
5	花 10 分钟读《安全五铁律》	第 23 章	能说出五条铁律

卡住了怎么办：把报错信息原样粘贴给智能体，问它："我是新手，请用大白话解释这个报错，并告诉我怎么解决。"——90% 的安装问题都能这样解决。

第 1 章 智能体到底是什么

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

先说结论：智能体（Agent）= 大模型（脑子）+ 工具（手脚）+ 记忆（经验）+ 流程（习惯），它像一个"什么都会一点、但没有经验的新员工"，需要你一步步引导、限制、验收。

从"大模型"到"智能体"

概念	一句话解释	类比
大模型（LLM）	读了海量书籍、网页、代码的"超级大脑"，一个字一个字预测着说话	刚毕业的学霸
智能体（Agent）	给大脑装上手脚、记忆和操作手册，能自主干活的系统	配齐工位的实习生
API KEY	大模型给你发的"门禁卡"，绑着账户余额，每次问答都扣钱	门禁卡
Base URL	给 Agent 写的"收件地址"，告诉它把请求发到哪个服务器	寄信地址
Skill	教 Agent"遇到某件事该怎么做"的说明书	标准菜谱
MCP	Agent 和外部工具之间的"USB 接口标准"，工具即插即用	USB 插口
多模态	不光读字，还能看图、听声、看视频的模型	五官齐全

小白注：后文看到这几个词时，可以这样翻译——**Token** = AI 按字数收费的单位（大约 1 个汉字 ≈ 1-2 个 token）；**终端 / CLI** = 那个打命令的黑窗口（Win+R 输入 cmd 回车就能打开）；**沙盒** = 给智能体围的"儿童围栏"，围栏外它碰不到。更多词见附录 D「小白术语表」。

为什么 Agent 要接入大模型？ 大模型是 Agent 的"脑子"——没有它，Agent 只是个按固定脚本办事的木偶；接上之后，它才能看懂需求、拆解任务、自己决定什么时候调用什么工具。

最重要的心法：把它当实习生

智能体能帮你解决大多数问题，但**不能解决所有问题**。他不是神，是实习生：需要你一步步引

导、限制、验收，而不是当甩手掌柜。判断不了结果对不对的任务，就不要让它独立完成。

新手高频问题速答

问题	答案
为什么我的智能体不能识别图片？	你接的是纯文本模型而不是多模态模型，换多模态模型即可
为什么它老是说英文？	直接对他说："以后所有回答都用中文。"
为什么我的智能体这么笨？	大概率是没装 Skill——没技能的新员工当然笨（见第 3 篇）
为什么它老是出错/乱删文件？	多半是因为开了"完全访问"权限。权限给最小化（见第 6 篇）
为什么本地部署的模型很卡？	模型参数超过硬件能力，或没做量化优化。直接说"我的本地模型很卡，帮我优化"
怎么选模型？	按任务选：DeepSeek 强于数学推理、GLM 强于代码、Kimi 强于文学长文、豆包强于语言与生图
如何省 Token？	普通任务用免费/低价模型（小米、豆包、GLM、Kimi），复杂任务再切强模型

智能体能帮你做什么

数据分析、写代码、设计前端、做 PPT、整理电脑、对接电商、解决电脑问题……入门建议先做两个小任务：① 按一份方案生成公司介绍 PPT；② 生成一个博客网站完整页面。做完你就知道"实习生"的能力边界了。

记住这点就够了：智能体是配齐工具和经验的大模型实习生——你教它、管它、验收它，它就真能替你干活。

第 2 章 AI 编程工具进化史与 2026 格局

基础版本 · V0.1

小白提示：本章是"行业新闻"，赶时间可以先跳过，等装好智能体再回来看。

难度：入门 | 更新时间：2026-08-14

大白话讲：AI 编程工具五年变了四次——从"帮你补代码"进化到"帮你交付任务"；2026 年选工具看定位：Claude Code 像终端里的长期搭档，Codex 像 OpenAI 生态的多端工程 Agent，国产工具负责便宜好用。

四次进化

阶段	时间	代表	AI 的角色
补全时代	2021	GitHub Copilot	更聪明的输入法：你写开头，它补后面
对话时代	2022	ChatGPT	问答伙伴：复制代码、粘贴报错、要答案
项目协作时代	2023–2024	Cursor	编辑器里的协作者：看文件、改函数、跨文件重构
工程 Agent 时代	2025–2026	Codex、Claude Code、Hermes	工程执行者：读项目→计划→改代码→跑命令→整理 PR

记住这点：**Copilot 帮你补代码，ChatGPT 帮你想代码，Cursor 陪你改项目，Codex / Claude Code 帮你执行工程任务。**

主流工具怎么选（2026-08 更新版）

工具	一句话定位	Windows 体验	2026 年 8 月状态
Claude Code	终端里的长期工程搭档，生态最成熟	原生	v2.1.22x，跨会话通信、Routines、动态工作流；社区报告 2.1.91–2.1.196 有后门风险，务必官方源安装并保持最新
Codex	OpenAI 生态的多端工程 Agent	官方桌面应用	已并入 ChatGPT 桌面 App：Chat（讨论）/ Work（交付成品）/ Codex（操作文件）三入口；GPT-5.6 三档模型

工具	一句话定位	Windows 体验	2026 年 8 月状态
Hermes	开源协作型智能体，会自行进化	Desktop 公测	v2026.4.x，开发商估值约 15 亿美元
Deveco Code（华为）	鸿蒙开发 + 通用 Vibe Coding		开源免费，内置 GLM-5.4 和小米模型，安全审计出色（9 分）
ZCode（智谱）	基本可平替 Codex		每天送 300 万 GLM-5.2 token
Trae（字节）	IDE 编码工具		国内免费，热门模型需排队
WorkBuddy	应用内技能包，一键安装		适合轻量办公，不适合大型开发

Codex 与 Claude Code 的区别：

- Claude Code 更像**终端里的长期协作搭档**：长读项目、持续追踪复杂任务、边讨论边修改，适合终端工作流；
- Codex 更像 **OpenAI 生态里的多端任务执行平台**：App/CLI/IDE/Web 四个入口切换，配合 ChatGPT 账号体系、GitHub PR 工作流、Skills 与插件生态；
- 两者没有绝对谁替代谁，最终看：模型能力、上下文处理、工具链、价格、团队习惯。

2026 年 7–8 月的重大变化：Codex 并入 ChatGPT

- **7 月中旬**：OpenAI 把 Codex 桌面 App 和 ChatGPT 桌面端合并，旧聊天界面改名 **ChatGPT Classic**；后因社区吐槽，"ChatGPT Codex" 又改回 "Codex"；
- 新 App 三个岗位：**Chat** 负责讨论、**Work** 负责交付成品、**Codex** 负责操作项目——口诀：**要答案用 Chat，要成品用 Work，要动文件用 Codex**；
- **7 月 9 日 GPT-5.6 发布三档模型**：Sol（旗舰，\$5/\$30）、Terra（均衡，GPT-5.5 性能一半价格）、Luna（最便宜最快，长文弱）；另有 Sol Ultra 高算力模式（默认并行 4 个子智能体，最高 16 个）；
- **额度规则**：Codex / Work / ChatGPT for Excel / Workspace Agents **共享同一套 Agent 额度**；普通 Chat 走聊天额度（如 Plus 每 3 小时 160 条）——所以"Chat 聊方案、Codex 干执行"是省钱根本；
- 模型选择口诀：**默认 Terra，简单 Luna，关键才用 Sol。**

说白了就是：先想清楚要用智能体做什么，再选工具；2026 年"选工具"的本质是"选生态"——深度用哪家就住进哪家。

第 3 章 2026 年 8 月速览：一页看懂最新格局

基础版本 · V0.1

小白提示：本章同样是"行业新闻"，可先跳过，不影响动手安装。

难度：入门 | 更新时间：2026-08-14

本质就一句：国产模型全面爆发、Codex 与 ChatGPT 合并、Skill 跨平台标准落地、数字人市场爆发——这一轮 AI 的主战场已经从"聊天"转向"干活"。

模型层：卷的不再是聪明，是干活便宜

事件	时间	要点
DeepSeek V4-Pro 正式版上线	2026-08-13	API 调价 8 月 17 日生效， 最高涨约 500% ；但编程测评第二、成本仅第一名 1/14，性价比仍在
Kimi K3 开源	2026-07	智能体终端中文测评居首（61 分）；月之暗面估值 500 亿美元
智谱 GLM-5 技术报告公开	2026-02	"全球大模型第一股"，代码能力强
豆包 2.0 / 小米 MiMo-V2.5-Pro	2026	豆包综合亮眼；MiMo 是"最便宜编程模型"之一
Anthropic Fable 5	2026	SWE-Bench Pro 80.3% 断层第一；Anthropic 估值 9,650 亿美元反超 OpenAI
GPT-5.6 (Sol/Terra/Luna)	2026-07-09	agentic coding 上限之战；官方 System Card 披露 Sol 有"谎报完成/刷分作弊"问题——别全信它说 done
Grok 4.5 / Muse Spark 1.1	2026-07	Grok 走性价比（\$2/\$6）；Meta 首次模型收费，主打多智能体编排（1M 上下文）

工具层：Windows 用户的好日子来了

- **Claude Code** v2.1.224 支持跨会话消息传递；桌面版重构 + Routines + 动态工作流；

- **Codex** 官方 Windows 桌面应用发布（2026-03）；CLI 支持接入任意开源模型；新增 /goal 长时域模式；
- **Hermes** Desktop 公测；**Ollama** v0.32 内置交互式智能体入口；
- 国产新秀：华为 Deveco Code、智谱 ZCode、CodeArts、字节 Trae。

协议与生态层：Skill 标准化 + 双协议

- **跨平台 Skill 标准已成事实**：以 agentskills.io 规范为代表的 SKILL.md 格式（YAML 元信息 + 渐进式披露）被多家平台采用——"一份 Skill 全平台通用"在 2026 年实现了（第 3 篇详解）；
- **MCP + A2A 双协议**：MCP（连工具）+ A2A（智能体之间协作）成为 2026 标准栈，帮企业搭"多智能体协同工作流"是新的接单蓝海。

市场层：数字人爆发

- AI 艺人/数字人广告报价已达 **25 万元/条**量级，"一张脸"的数字人经济进入大众视野——第 5 篇"数字人代运营"路径现在是 2026 最火方向之一；
- AI 副业从"卷信息差"转向"卷交付质量"："AI 初稿 + 人工把关"才是能持续赚钱的模式。

最后提一句：2026 年的 AI 赛道像极了 2016 年的抖音——所有人都知道这是未来，但大多数人没有当回事。别等完美，先上车。

顺序不能错：**先申请 API KEY（外部账号）→ 再配本地环境（Git / Node / Python）→ 最后装智能体**。90% 的"装不上"问题，根源都在环境这一步。

第 4 章 准备：API Key 与环境三件套

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

核心就一点：API KEY 是门禁卡、Base URL 是收件地址；Git、Node.js、Python 是所有智能体工具的本地地基——三个软件都装对，智能体才跑得起来。

第一步：申请 API KEY（2026-08 更新版渠道表）

小白注：API KEY = 大模型平台发给你的"门禁卡"，一串很长的字符，代表你的账户和余额，**千万别截图发群里**；Base URL = 告诉智能体"把请求寄到哪个地址"的收件地址。申请 Key 时：注册平台账号 → 找到"API Key / 密钥管理" → 点"创建" → 复制保存。

渠道	地址	说明（2026-08 更新）
DeepSeek	https://platform.deepseek.com/	V4-Pro 正式版已上线，8 月 17 日起新定价生效（涨幅最高约 500%），编程性价比仍居前列，申请后留意账单
豆包（火山方舟）	https://www.volcengine.com/product/ark	豆包 2.0 系列表现亮眼，有免费额度
小米 MiMo	https://platform.xiaomimimo.com/console/balance	填邀请码送 10 元额度；MiMo-V2.5-Pro 是"最便宜编程模型"之一
智谱 GLM	https://open.bigmodel.cn/	GLM-5 系列已发布，代码能力强；ZCode 每天送 GLM-5.2 token（见第 7 章）
月之暗面 Kimi	https://platform.moonshot.cn/	Kimi K3 已开源，文学/长文本强
NVIDIA Build（白嫖）	https://build.nvidia.com/settings/api-keys	免费额度，有调用次数限制；不适合接 Codex
Agnes AI（白嫖）	https://agnes-ai.com/	免费额度，有调用次数限制

常用 Base URL（配智能体时填）：

- 小米：<https://api.xiaomimimo.com/v1>

- DeepSeek: <https://api.deepseek.com>
- 本地 Ollama: <http://localhost:11434> (API KEY 随便填)

友情提示：不建议使用"中转站"——虚标、降智、跑路是常态。非必要不用中转；复杂任务请擦亮眼睛，不要被低价吸引。有条件就用官方渠道。

补充：不同场景怎么选模型？

你的场景	推荐模型	为什么
写代码 / 改代码	Claude Sonnet / DeepSeek-Coder	代码能力强、理解上下文准
写文章 / 做文案	GPT-4o / 豆包	中文表达自然、风格灵活
省钱跑日常任务	DeepSeek / 智谱 GLM / 小米 MiMo	国内平台有免费额度，单价低
本地离线 / 隐私优先	Ollama + Llama3 / Qwen	数据不出本机，不花钱
复杂推理 / 长文档分析	Claude Opus / GPT-4o	上下文窗口大，推理深

小白建议：先别纠结选哪个——国内平台注册就送免费额度，每个都试一下，哪个回答你满意就用哪个。用熟了再考虑付费。

第二步：安装 Git (Windows 小白版)

1. 下载: <https://git-scm.com/download/win>，点页面中部灰色卡片 "Standalone Installer — Git for Windows / x64 Setup";
 - 不点 "Portable" (绿色版易出问题)；不点 "ARM64" (特定电脑专用)；
2. "选择目标位置"那一步：**确认路径是 C:\Program Files\Git 后直接 Next，不要改路径**；
3. "Choosing the default editor"那一步：确认 Nano 默认选中，直接 Next；
4. 其余全默认。验证：Win+R → cmd → 输入 `git --version`，显示 `git version 2. xx. x. windows. 1` 即成功。

怎么算装对了：看到 `git version 2. xx. x. windows. 1` 就是成功。**常见报错：**提示"不是内部或外部命令"→说明当前窗口是安装前打开的，**关掉黑窗口重新打开一个再试**；还不行就重启电脑。

第三步：安装 Node.js

1. 下载：<https://nodejs.org/zh-cn/download/>，点左侧 **LTS** 的 .msi（不点右侧 Current）；
2. 到 "Custom Setup" 界面**确认勾选 "Automatically install the necessary tools"**（默认已勾，别取消）；
3. 安装中弹出的黑色窗口是自动装编译工具，**别关**，等它跑完（约 3-5 分钟）；
4. 验证：`node -v` 显示 `v20.xx.x` 或更高、`npm -v` 显示 `10.xx.x` 或更高。

怎么算装对了：`node -v` 和 `npm -v` 都能显示版本号。 **常见报错：**提示"不是内部或外部命令"→ 同样**关掉黑窗口重新打开一个再试**（安装后必须开新窗口才生效）。

第四步：安装 Python

1. 下载：<https://www.python.org/downloads/windows/>，找 **Python 3.11.x 或 3.10.x**（2026 年部分新库已支持 3.12+，但 3.11 兼容性最稳），点 "Download Windows installer (64-bit)";
2. 打开安装包**第一件事：勾选最下方 "Add Python to PATH"**（这个选项默认不勾，必须手动勾！）；
3. 点 "Install Now"。验证：`python --version` 显示 `Python 3.11.x`；若提示"不是内部或外部命令"= 忘了勾 PATH，卸载重装。

怎么算装对了：`python --version` 显示版本号。 **常见报错：**① 提示"不是内部或外部命令"→ 安装时没勾 "Add Python to PATH"，先开新窗口重试，不行就卸载重装（重装时记得勾）；② 弹出微软商店 → 也是没勾 PATH 的表现，同样处理。

划个重点：API KEY + 三个环境软件（Git/Node/Python）各一步验证命令，全部通过再装智能体。

第 5 章 Claude Code 安装与上手

基础版本 • V0.1

难度：入门 | 更新时间：2026-08-14

说穿了就是：一条 npm 命令装好 Claude Code；它像"长期待在终端里的 AI 工程搭档"，核心习惯是：改完必看 diff、复杂任务先 plan、权限保持"请求批准"。

终端版安装（三步）

1. 装好 Node.js 和 Git（第 4 章已做）；
2. PowerShell 或 CMD 输入：

```
npm install -g @anthropic-ai/claude-code
```

3. 启动：claude。如提示脚本无法运行，先执行 Set-ExecutionPolicy RemoteSigned。

小白注：看到"无法加载文件……因为在此系统上禁止运行脚本"别慌——在**管理员身份**打开的 PowerShell 里输入 Set-ExecutionPolicy RemoteSigned，回车后输入 Y 确认，再重新运行 claude 即可。

安全提醒（原教程 + 2026 更新）：社区曾报告 Claude Code **2.1.91-2.1.196** 版本存在后门风险。请只从官方 npm 源安装、保持最新版本（当前 v2.1.22x 以上），不要下载来路不明的修改版。

桌面版

下载地址：<https://code.claude.com/docs/en/desktop>（2026 桌面版已全面重构，支持 Routines 等新特性）。桌面版需要官方账号，请按第 6 篇的合规注意事项使用官方渠道。

常用命令速查表

指令/快捷键	作用	场景
/compact	压缩对话上下文	对话过长 token 超限时
/clear	清空对话历史	切换完全不同任务时
claude -c	启动时恢复上一次对话	次日继续昨天的工作
Ctrl+B	任务挂到后台	AI 后台编译，前台讨论下一步
ESC×2 (/rewind)	回退/总结	AI 改错了代码，快速撤销
/tasks	管理后台任务	检查后台编译是否完成
Ctrl+T	显示/隐藏任务面板	查看任务进度
Alt+V	粘贴图像（Windows）	截图给 Claude 分析 UI/报错
/memory	编辑 CLAUDE.md	修改项目规则/个人偏好
/init	自动生成 CLAUDE.md 初稿	新项目首次接入
/hooks	配置工具钩子	写文件后自动格式化
Shift+Enter	输入框内换行	输入多行代码时
Shift+Tab	切换三种工作模式	Plan Mode Accept Edits

2026 新增：Claude Code v2.1.224 起支持跨会话消息传递——多个终端里的 Claude 实例可以互相发消息协调工作，适合多线任务并行。

推荐新手 workflow

步骤	操作	目的
1	进入项目目录，启动 claude	让它以当前项目为工作区
2	/init 生成 CLAUDE.md	建立项目规则
3	描述任务（复杂任务先说"先出方案，不要直接改"）	先对齐方向
4	检查 diff	看它实际改了什么
5	git commit 保存满意版本	可回滚
6	/compact 对话长了就压缩	省 token 保上下文

带走这句话：Claude Code 是终端搭档——记住"改完必看 diff、复杂先 plan、权限最小化"三件事就够了。

第 6 章 Codex 安装与上手

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

一句话讲明白：Codex 有四种入口（App / CLI / IDE / Web），新手从 App 开始最简单；2026 年它已并入 ChatGPT 桌面 App——要答案用 Chat、要成品用 Work、要动文件用 Codex。

认识 Codex：从代码补全到工程 Agent

Codex 不是"又一个写代码的 ChatGPT"，而是能进入项目、读文件、理解上下文、定计划、改代码、跑命令、整理 PR 的**工程执行者**。它适合：目标明确、范围可控、结果能验证、失败能回滚的任务；不适合：生产数据库、支付核心、大规模迁移等你自己都验收不了的任务。

四种入口怎么选

入口	适合谁	需要准备
Codex App（桌面版）	小白、想要图形界面的人（Windows 官方应用已发布）	ChatGPT 账号
Codex CLI	稍微懂终端的人	终端、Git、项目环境
IDE Extension	用 VS Code / Cursor / Windsurf 的人	编辑器 + 插件
Codex Web / Cloud	想让 Codex 远程处理 GitHub 项目的人	GitHub 仓库

App 安装与上手（新手最推荐）

- 官方页面下载 Windows/macOS 版（Mac 按芯片选 Apple Silicon 或 Intel，别选错）；
- 第一次打开：登录 → 选**项目目录**（选一个干净的练习目录，不要 C 盘根目录、不要重要项目）；
- 理解三个核心概念（类比表）：

概念	简单来说
项目（Project）	一个代码文件夹 = 一家公司
Thread	项目里的一个任务对话 = 一个员工（一个任务开一个 thread，别全塞一起）
Review Pane / Diff	"作业检查区"——绿色新增、红色删除， 每次完成先看 diff，再看它的总结

小白注：diff = 智能体改动的"前后对照表"：绿色行 = 新加的，红色行 = 删掉的。看不懂就全选复制，问它："用大白话解释你改了什么。"

- 4. 推荐第一个任务：请帮我做一个简单网页：黑色背景，中间显示 Hello, Codex，字体白色，水平垂直居中，只使用 HTML 和 CSS；
- 5. 完成流程：输入任务 → 等它改文件 → 打开 review pane → 看 diff → 没问题再继续。

沙盒权限（保命核心）：

权限选项	与沙盒的关系	建议
请求批准	有围栏，越界操作先问你	新手推荐
替我审批	系统帮你自动判断一部分审批	可接受
完全访问权限	放开围栏，可执行任何操作	新手不要开

推理强度四档：低（改文案/改颜色，省额度）→ 中（日常开发，默认）→ 高（复杂 bug、重构）→ 超高（架构分析、反复修不好的 bug）。**不要所有任务都开最高。**

计划模式：复杂任务先开计划模式（或输入 /plan）——让 Codex 先出工作计划，你确认后再动手，能查漏补缺。

CLI 安装与登录

```
npm install -g @openai/codex # 安装
codex --version # 验证
codex # 启动（首次提示登录）
codex login # ChatGPT 账号登录（小白首选）
codex login status # 查看登录状态
codex logout # 退出（换账号/公共电脑）
```

API Key 登录（开发者/自动化）： \$env:OPENAI_API_KEY="你的Key" 然后 \$env:OPENAI_API_KEY | codex login --with-api-key。Key 绝不能写进代码/截图/README/Git 仓库。

CLI 必会命令（10 条）

命令	作用	场景
codex	启动 CLI	进入项目后使用
cd 项目目录 + codex	指定工作区（或 codex -C 路径）	别在 C 盘根目录/桌面启动
codex exec "任务"	一次性执行任务	自动化、快速检查

命令	作用	场景
codex resume --last	恢复最近会话	接着上次任务继续
codex archive	归档会话	任务完成收起来
codex doctor	环境诊断	启动/登录异常时
codex update	更新版本	需要升级时
codex mcp list	查看 MCP 工具	配置 MCP 后检查
/diff	查看改动	修改后必看
/model、/permissions、/plan、/init、/compact、/quit	斜杠命令	见下

推荐新手组合： `codex --sandbox workspace-write --ask-for-approval on-request`（项目内可工作、敏感操作先问你）。

危险命令红线（看到先喊停）：

命令/参数	为什么危险
<code>--sandbox danger-full-access</code>	放开文件和网络限制
<code>--dangerously-bypass-approvals-and-sandbox</code> （别名 <code>--yolo</code> ）	跳过审批和沙盒
<code>--ask-for-approval never</code>	操作不再问你
<code>rm -rf、git reset --hard、git clean -fd、curl xxx sh</code>	可能删库/丢改动/执行远程代码

看到这些，先问 Codex："请解释这条命令的作用、风险，以及有没有更安全的替代方案。"

2026 年 7 月后的变化：Codex 并入 ChatGPT

变化	要点
三入口合并	桌面 App 里 Chat（讨论）/ Work（交付成品）/ Codex（操作文件）；旧聊天改名 ChatGPT Classic
模型三档	默认 Terra ，简单批量 Luna ，关键任务 Sol ；Sol Ultra 多智能体并行（默认 4 个子智能体，最高 16）
额度共享	Codex/Work/Excel/Workspace Agents 共享 Agent 额度；Chat 走聊天额度—— Chat 聊方案、Codex 干执行，最省钱
官方 Prompt 四要素	Goal（目标）/ Context（上下文）/ Output（输出格式）/ Boundaries（边界） ——重要任务写清这四点
Steer / Queue	任务运行中：Enter = 插队补充，Tab = 排队等下一轮；Ctrl+M 语音听写

变化	要点
退役节奏	GPT-5.4 已于 7-23 退役，旧 Codex 模型 8 月底退场——项目尽早迁移到新模型名

国内使用方案（2026-08 实测总结）

官方 GPT 账号登录最稳；接第三方 API 属进阶操作。三种方案：

方案	适合谁	注意
手动改 ~/.codex/config.toml	想理解底层的人	base_url 只写到 /v1；wire_api="responses" 别改；requires_openai_auth=false；Mac 需从终端 open -a Codex 启动
Codex++	桌面 App 用户 (最丝滑)	图形化管理，支持插件；第三方工具，Codex 更新后可能需适配
CCX + CC Switch	多供应商重度用户	网关做协议转换（Chat Completions → Responses），组件多需理解端口链路

排错口诀：切换没生效→完全重启+检查 model_provider 名称；认证错误→查 Key 与环境变量；路径错误→base_url 只到 /v1；国产模型无响应→上游是否支持 Responses API。

IDE 插件与 Web/Cloud（一句话版）

- **IDE 插件：**VS Code/Cursor 扩展市场搜 Codex 安装，登录 ChatGPT 账号，侧边栏即可边写边问，适合日常编码；
- **Web/Cloud：**chatgpt.com/codex 连接 GitHub 仓库，云端跑任务、生成 PR——适合团队协作；只授权需要的仓库、不要让 Codex 自动合并 PR、不要把 Key 写进任务。

收个尾：Codex 从 App 入门，把"项目/thread/diff/沙盒"四个概念搞懂，你就拿到了工程 Agent 的钥匙；2026 年记得用"Chat 聊方案 + Codex 干执行"省钱。

第 7 章 Hermes 与国产工具

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

先讲重点：Hermes 是会"自行进化"的开源协作智能体，微信一键对接；国产阵营里 Deveco Code 免费内置 GLM-5.4 + 小米模型，ZCode 每天送 300 万 token——没有魔法也能玩得转。

Hermes 安装（终端版）

- PowerShell 输入：`irm https://hermes-agent.nousresearch.com/install.psl | iex;`
- 装好后重新打开终端：`hermes model` → 选模型厂家 → 填 API KEY → 填模型 URL → 选模型 → 重启；
- 对接微信：**直接在对话框跟 Hermes 说，它会给你设置并输出二维码链接，扫码即可；
- 装 Skill：**对话框说"安装 find-skills"即可自动按需搜索安装；知道名字的直接报名字让它装；也可以下载后用 CC Switch 导入或自己写（第 3 篇）。MCP 同理；
- Hermes 是协作类智能体，会根据你的使用习惯自行进化。

2026 更新：**Hermes Desktop 桌面版已公测**（此前"桌面版不完善"的说法已过时）；命令文档：<https://hermesagent.org.cn/docs/reference/cli-commands>

国产新秀（2026 免魔法）

工具	安装/启动	特点
Deveco Code （华为）	<code>npm install -g @deveco/deveco-code → deveco</code>	启动→选第一个登录华为账号→返回终端→点第二个→第一个（勾选框记得勾）。专为鸿蒙开发设计，实测也能做通用 Vibe Coding； 安全审计 9 分 ；免费内置 GLM-5.4 和小米所有模型
ZCode （智谱）	https://zcode.z.ai/cn	基本平替 Codex，无电脑控制工具；每天 300 万 GLM-5.2 token
CodeArts （华为云）	https://codearts.huaweicloud.com/	IDE 编码工具，可白嫖试用
Trae （字节）	官网下载	国内免费，热门模型需排队；搭配 TestSprite（付费）

CC Switch（模型配置管理）

作用：给 Claude Code 等 Agent 配置模型（**不要给 Codex 配置**，会冲突）。右上角扳手 = Skill 管理（下载一次多 Agent 通用），另有提示词/会话/MCP 管理。

1. 下载：<https://github.com/farion1231/cc-switch/releases> → Windows 选 CC-Switch-v3.16.x-Windows.msi；
2. 顶部选 Agent → 点加号选模型 → 填 API KEY → 高级选项获取模型列表 → 选模型 → 按需勾 1M 上下文/最大思考 → 添加并启用。

VS Code（选配）

下载 <https://code.visualstudio.com/> → 装中文语言包 → 装 Claude Code 插件（前提：终端版已配好）。用法去 B 站搜。

别忘了一点：想要"会进化"的玩 Hermes；想要免费国产全家桶装 Deveco Code 或 ZCode；CC Switch 管模型切换，但别和 Codex 混用。

第 8 章 Ollama 本地部署

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

说白了：Ollama 把开源大模型跑在你自己电脑上——数据不出本机、免 API 费，但吃硬件；先查硬件再选模型，卡了就让它帮你优化。

安装与选模型

- 1. 下载：<https://ollama.com/> 一路安装；
- 2. 装完后**让智能体查询你的硬件信息**（CPU/内存/显卡/显存），结合硬件推荐模型规模；
- 3. 响应慢就说："模型响应太慢了，请优化。"让它做量化/配置适配，能提升数倍速度。

显存参考：8GB 以下 → 7B 及以下量化模型；16GB → 13B/14B 量化；24GB+ → 32B 级模型。跑不动别硬跑，换小的。

常用命令

命令	说明	示例
ollama run	运行模型（不存在自动拉取）	ollama run llama3
ollama pull	只下载不运行	ollama pull mistral
ollama list	列出本地模型	ollama list
ollama rm	删除模型释放空间	ollama rm llama3
ollama cp	复制模型（测试用）	ollama cp llama3 my-model
ollama create	按 Modelfile 创建自定义模型	ollama create my-bot -f ./Modelfile
ollama show	查看元数据/参数	ollama show --modelfile llama3
ollama ps	运行中的模型及显存占用	ollama ps
ollama push	推送自定义模型	ollama push my-username/my-model
ollama serve	启动 API 服务（通常后台自动）	ollama serve
ollama help	查看任意命令帮助	ollama help run

接入智能体

- 本地接入 URL：http://localhost:11434（先部署模型再用）；API KEY 随便填；
- 量化可大幅降显存提速度，社区有现成量化模型；
- Ollama v0.32 起内置**交互式智能体入口**，命令行体验升级。

临走记住：本地部署 = 隐私 + 免费 + 吃硬件；按显存选模型、让它自己优化配置，别小马拉大车。

不管用 Claude Code、Codex、Hermes 还是 WorkBuddy，所有智能体都有一个共同的核心概念：**Skill**。搞懂 Skill，你用任何智能体的效率都能翻倍。**使用任何 Skill 前，请先检查是否存在恶意代码**（检查方法见第 23 章）。

第 9 章 Skill 是什么、为什么

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

先给你交个底：Skill 就是写给 AI 的"标准菜谱"——说清楚什么时候用、怎么做、做完交出什么；没有 Skill 每次从零摸索，有了 Skill 按成熟方法来。

标准菜谱的比喻

你开了个餐厅，雇了个新厨师。你不能每次都手把手教他做菜，所以你写了一份**标准菜谱**：什么时候做、用什么料、按什么步骤、出什么成品。Skill 就是写给 AI 的"标准菜谱"。

一份合格的 Skill 包含三样东西：

- 什么时候用：**如"当用户说要做视频的时候"。智能体靠这段判断要不要翻开说明书；
- 具体怎么做：**第一步干什么、用什么工具、参数设多少——越具体越好；
- 容易出什么错：**哪些操作会搞砸、怎么避免、怎么修——这部分是经验，比步骤更值钱。

三个常见误区（新手必看）

误区	真实情况
Skill 就是写个提示词	不对。提示词是"临时对话"，Skill 是"可反复使用的标准流程"——"今天帮我炒个菜" vs "写一本标准菜谱"
Skill 要写得详细复杂	不对。越简洁越聚焦执行越好——菜谱写 3 页没人看，写 1 页步骤清晰才是好菜谱
Skill 要解释"为什么"	不用。Skill 是"下命令"不是"写论文"，AI 只需要知道怎么做

没有 Skill vs 有 Skill

没有 Skill	有 Skill
每次都要重复交代（分辨率、帧率、工具都要重说）	不用重复交代，说一句它自己知道怎么做
容易犯同样的错（这次纠正了下次还犯）	注意事项写在里面，每次执行前都会看
质量忽高忽低（每次方法可能不一样）	步骤固定、参数固定、质量稳定

没有 Skill	有 Skill
经验带不走	可以分享给同事，装上就能用

记住这个： Skill 是智能体的操作说明书，是"有经验的助理"和"新来的助理"的区别。

第 10 章 平台 Skill 与 2026 跨平台标准

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

讲透就一句：各平台叫法不同但本质一样；2026 年以 agentskills.io 为代表的 SKILL.md 跨平台标准已经落地——一份 Skill，多平台通用。

各平台 Skill 对照（2026 更新版）

平台	叫什么	文件格式	在哪找
Hermes	Skill	SKILL.md	hermes skills browse / 对话框
Claude Code	Skills / CLAUDE.md	Markdown	项目根目录、~.claude/skills
Codex	AGENTS.md / Skills (/skills 或 \$ 调用)	Markdown	项目根目录 .agents/skills / 插件市场
WorkBuddy	技能包	内置/插件	应用内市场
Cursor	.cursorrules	纯文本	项目根目录
通用标准	Agent Skills	SKILL.md (YAML frontmatter + 渐进式披露)	agentskills.io / GitHub 社区

2026 标准 SKILL.md 长这样

```
---
name: running-tests
description: "当用户要跑测试的时候用这个Skill"
---

# 以下是正文 — 告诉AI具体怎么做

## 什么时候用
当用户说"跑测试"、"运行测试"、"测试一下"的时候
```

```
## 什么时候不用
当用户只是问“什么是测试”（只是聊天，不需要真正执行）

## 具体步骤
1. 先找到项目里的测试文件
2. 执行 pytest 命令
3. 把结果整理成表格返回

## 出错怎么办
- 如果没有测试文件 → 告诉用户“没找到测试文件”
- 如果测试失败 → 列出失败的用例和原因
```

--- 上面是元信息（frontmatter：名字 + 触发描述，决定"什么时候用"），下面是正文（告诉 AI 怎么做）。描述写清楚，AI 才能在对的时候自动调用它。

插件、Skill、MCP 三者关系（先记这张表）

小白注：MCP 全称 Model Context Protocol，你可以把它理解成智能体的"USB 接口"——不管什么牌子的鼠标键盘，插上就能用。MCP 让智能体能连数据库、搜网页、读文件，不用每次都自己写对接代码。

对比	插件 Plugin	Skill	MCP
一句话	能力安装包	一套固定工作方法	连接外部工具的接口
解决什么问题	安装、打包、分发能力	同类任务"怎么做"	"连接什么工具或数据"
类比	工具箱	工具箱里的说明书	给工具箱接电的插座
举例	GitHub 插件、Chrome 插件	README Skill、代码 Review Skill	数据库 MCP、文档 MCP

记住这点："怎么做"的问题用 Skill，"连接什么工具"的问题用 MCP；插件可以把 Skill 和 MCP 打包成更容易安装的能力包。

推荐社区 Skill（2026 实测口碑）

Skill	作用	地址
Superp owers	给 Coding Agent 装一整套软件开发方法论（先澄清需求→写规格→实现计划→TDD 推进），适合 Codex/Claude Code/Cursor 等	https://github.com/obra/superpowers
baoyu- skills	宝玉整理的内容创作类：小红书图文、文章配图、公众号发布、X/微博发布、网页转 Markdown 等	https://github.com/JimLiu/baoyu-skills
Agent Reach	给本地 Agent 装"联网能力"：网页、YouTube、RSS、GitHub、X、B站、小红书等	https://github.com/Panniantong/Agent-Reach
find- skills	"找 Skill 的 Skill"，问一句"有没有某某 Skill"它会帮你搜索发现安装	https://github.com/vercel-labs/skills/tree/main/skills/find-skills

落地就一句：叫法不重要，本质都是说明书；2026 年 SKILL.md 已是跨平台通用格式，插件管分发、Skill 管方法、MCP 管连接。

第 11 章 怎么找、怎么判断一个好 Skill

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

抓重点：先想清楚你反复交代的是什么，再去平台和社区找；判断好坏看五条：描述清、有踩坑、步骤具体、更新勤、评价好。

第一步：想清楚你要解决什么问题

你经常让智能体做的事是什么？哪些事每次都要重复交代？哪些事它总做不好？想清楚再找。

第二步：在对应平台搜索

平台	怎么找
Hermes	hermes skills search 关键词 或直接说"帮我搜有没有做 XXX 的 Skill"; hermes skills browse 逛市场
Claude Code	GitHub 搜 "claude skills"、"CLAUDE.md template"
Codex	插件市场 //skills/ GitHub 搜 "codex skills"; codex plugin list 看插件
WorkBuddy	应用内"技能"页面一键安装

第三步：社区找

GitHub 搜 "ai agent skill"、"claude code rules"、"codex agents.md"; Reddit、V2EX、掘金、抖音都有大量推荐。

判断一个 Skill 好坏的五条标准

判断点	好 Skill	差 Skill
描述清不清楚	写清"什么时候用、能干什么"	描述模糊 ("通用工具")
有没有踩坑经验	有"注意事项/常见问题" (作者真用过)	没有踩坑部分, 实用性存疑
步骤具体不具体	具体命令、具体参数	只写"用 XX 工具处理"

判断点	好 Skill	差 Skill
更新频率	近期有更新（尤其涉及 API/工具版本）	半年没动，可能已过时
评价和使用量	下载量高、评价好	无评价数据

实操要点：找 Skill 先问自己"我在反复教它什么"；选 Skill 先看"注意事项"——有踩坑经验的才可信。

第 12 章 怎么写自己的 Skill

基础版本 · V0.1

难度：进阶 | 更新时间：2026-08-14

最核心的一点：写 Skill 的关键三点——描述清楚什么时候用、步骤具体到命令、把踩坑经验记下来；先让它能跑通，再慢慢完善。

什么时候该写

- 你重复教过它三次以上——每次都交代工具、参数、格式，写成 Skill 一劳永逸；
- 它犯了一个你觉得不该犯的错——纠正后写进注意事项，它就不会再犯；
- 你有一个多步骤工作流程——如"写文案 → 做幻灯片 → 抓帧 → 合成视频"，写成 Skill 不漏步骤；
- 你的项目有特殊规范——"注释用中文""commit 用 Conventional Commits"等，写成项目级 CLAUDE.md / AGENTS.md。

通用模板

标题（做什么事）

什么时候用

- 触发条件 1：用户说「XXX」
- 触发条件 2：用户要做 XXX
- 不要用的场景：XXX

环境和工具

- 需要安装什么：XXX
- 需要什么权限：XXX
- 用什么工具：XXX

操作步骤

1. 第一步：具体做什么，用什么命令
2. 第二步：具体做什么，参数设多少

注意事项（踩坑经验）

1. 不要做 XXX，因为会 YYY

2. 一定要做 XXX，否则会 YYY
3. 如果遇到 ZZZ 错误，用 WWW 方法解决

验证结果

- 检查 XXX 是不是正确
- 跑一遍 XXX 确认没问题
- 预览一下最终效果

真实例子：项目级规范（CLAUDE.md / AGENTS.md）

项目规范

技术栈

- Python 3.11
- FastAPI + SQLAlchemy
- pytest 测试框架

代码风格

- 注释用中文
- 变量名用英文驼峰
- 函数名用英文下划线
- 每个函数必须写 docstring

提交规范

- commit message 用英文
- 格式：type: description (如 feat: add user login)
- type 只能是 feat/fix/docs/refactor/test/chore

测试要求

- 新功能必须写单元测试
- 测试文件放在 tests/ 目录
- 运行命令：pytest tests/ -v

注意事项

- 不要修改 alembic/ 下的迁移文件
- 不要在 main.py 里写业务逻辑
- 数据库密码必须从环境变量读取

放在项目根目录，智能体每次启动都会读，写代码时自动遵守。（Codex 里可以用 `/init` 生成，也可以直接对它说"把项目初始化成 AGENTS.md"。）

写好 Skill 的 5 条核心原则

- 1. **名字要准确**——小写字母 + 连字符： `running-tests test-helper`；
- 2. **只做一件事**——一个 Skill 只管一件事，想要多个能力就写多个；
- 3. **说清"什么时候用/不用"**—— "当用户要求跑单元测试或执行 `pytest` 时使用；只是问概念时不使用" "用于测试相关场景" (AI 会乱猜)；
- 4. **步骤具体到能直接执行**——不写"适当处理": "运行 `pytest --tb=short`；返回码非 0 则提取最后 20 行错误；输出通过数/失败数/失败用例名" "运行测试、检查结果、给出报告"；
- 5. **写清楚"出错怎么办"**——找不到文件、超时、权限不足分别怎么回复，异常覆盖才是高手。

该管多松多紧？

任务类型	管多紧	举例	怎么写
创造性任务	松一点	写代码审查意见、写文章	给方向和原则
半固定任务	中等	生成报告、写周报	给模板结构，内容可灵活
不能出错的任务	管死	数据库迁移、部署上线	每步写死命令

就像带小孩：画画课随便画（松），做数学题按公式（中），过马路必须牵手（紧）。

从零写一个 Skill 的完整流程

先让 AI 裸干一次 → 记录哪里翻车 → 写最小 Skill → 测试验证 → 慢慢补充完善

- 1. **先不写，让 AI 直接干一次**：记录哪里不好——理解错需求？步骤乱？格式不对？
- 2. **设计 3~5 个测试场景**：每个场景写清"输入什么、期望输出什么"，这就是以后的"考卷"；
- 3. **写最小版本**：只写能解决翻车问题的规则，先跑通；
- 4. **拿"考卷"验证**：没通过继续改；
- 5. **逐步补充边界**：空输入怎么办？文件不存在怎么办？

先让它能用，再让它好用。不要一上来就追求完美。

聪明人的做法：让 AI 帮你写

- 1. 先让它干一遍活（正常对话完成一次真实任务）；
- 2. 让它复盘："你刚才做了哪几步？""哪些是每次都要做的固定流程？""哪些地方你不太确定？""什么情况下不该用这个流程？"
- 3. 把复盘结果整理成 `SKILL.md`（Codex 里直接用 `$skill-creator`，它会问你用途/触发场景/输出格式/限制并生成）；
- 4. 测试、调整、再测试。

就像：让员工自由发挥干一次 → 让他总结 → 你帮他写成标准流程 → 以后所有新员工照着做。

新手避坑清单

坑	为什么是坑	怎么避免
用 Windows 路径 C:\xxx\	换系统就炸	统一用 xxx/yyy/ 格式
给太多选择	AI 不知道选哪个，结果不稳定	直接指定："默认用 PostgreSQL"
写入会过期的信息	如"2026 年 5 月前用旧 API"	过期信息放单独文件，标注"已废弃"
同一个东西叫不同名字	一会儿"接口地址"一会儿"API URL"，AI 混乱	全文统一一个名称
Skill 写得像论文	AI 不需要知道"为什么"	删掉解释性文字，只留操作指令
一个 Skill 干太多事	经常丢步骤或搞混	拆成多个 Skill

核心别丢：写 Skill 不是高深技术——把你做的事情写成 AI 能照着做的说明书。你能教会一个新员工，就能写出一个好 Skill。

第 13 章 完整示例与文件组织

基础版本 · V0.1

难度：进阶 | 更新时间：2026-08-14

先定个调：一个完整 Skill = 一个文件夹 + 一个 SKILL.md (≤500 行)，长资料拆进 references/；写好 description 让 AI 自动判断何时调用。

完整示例：自动整理下载文件夹

```
---
name: organize-downloads
description: "整理下载文件夹。当用户说'整理下载'、'清理下载文件夹'时触发。"
---

## 什么时候用
- 用户说“整理下载”、“清理下载文件夹”、“帮我归类下载的文件”

## 什么时候不用
- 用户只是问“下载文件夹在哪里”（只问位置，不需要整理）

## 输入
- 下载文件夹路径（默认 ~/Downloads）

## 输出
- 整理报告：移动了多少文件、每个类别有多少个

## 具体步骤
1. 扫描下载文件夹里所有文件
2. 按文件类型分类：
  - 图片（.jpg .png .gif .webp）→ 移到 ~/Downloads/图片/
  - 文档（.pdf .doc .docx .txt）→ 移到 ~/Downloads/文档/
  - 视频（.mp4 .mkv .avi）→ 移到 ~/Downloads/视频/
  - 压缩包（.zip .rar .7z）→ 移到 ~/Downloads/压缩包/
  - 其他类型 → 保持不动
3. 如果目标文件夹不存在，先创建
4. 如果目标位置已有同名文件，在文件名后加日期后缀
```

5. 输出整理报告

出错怎么办

- 文件正在被占用 → 跳过，报告中注明“跳过（被占用）”
- 权限不足 → 跳过，报告中注明“跳过（无权限）”
- 移动失败 → 不删除原文件，报告中注明失败原因

注意看这个示例：名字准确、描述有触发词、"用/不用"写清、步骤能直接执行、错误也覆盖了——这就是合格 Skill。

Skill 文件怎么放

```
skills/  
  organize-downloads/  
    SKILL.md # ← 主文件，必须有  
  running-tests/  
    SKILL.md  
  references/ # ← 补充资料（可选）  
    pytest-cheatsheet.md  
  scripts/ # ← 可选脚本  
  assets/ # ← 可选模板/资源
```

- 每个 Skill 一个文件夹，文件夹名 = Skill 名；SKILL.md 放核心内容（建议 ≤500 行）；
- 长资料放 references/，AI 需要时自己去读——这就是 2026 标准的**渐进式披露**：主文件精简，细节拆出去；
- 放置位置：项目内 .agents/skills/（项目专属）→ 用户级目录（个人通用）→ 插件（打包分发）。

怎么让它"自动学习"

- Hermes 等有"自动保存 Skill"功能：纠正一次，它自动保存，下次直接用；
- 没有这功能就手动：① 发现做错 → 告诉它"应该 XXX"；② 说"把这个经验保存下来，以后遇到同样的事直接用"；③ 下次验证有没有用上，没用上就检查 Skill 是否被正确加载。

Skill 的未来（2026 已部分实现）

2025 年预言	2026 年 8 月现状
Skill 市场会越来越大	已实现：各平台市场 + GitHub 社区爆发
Skill 会越来越专业	进行中：行业专用 Skill（财务、法律、医疗）不断出现
跨平台 Skill 会出现	已实现：Agent Skills 标准成为跨平台通用格式
AI 会自己写 Skill	进行中：从"自动保存纠正"到"观察工作流自动生成"

走之前记住：Skill = 说明书。描述清触发条件、步骤写到命令级、踩坑经验全记下；主文件保持精简，细节拆出去。

小白提示：本篇讲"软件公司里怎么协作干活"，纯新手可先跳过，等你开始接活或入职时再读；但第 19 章的提示词模板库值得现在收藏。

写给刚入行的新人。这不是标准答案，只是大多数项目的默认套路。实际情况可能更乱，做好心理准备——本篇章既讲"理想流程"，也讲"现实套路"，还附一套可直接复制给智能体用的提示词模板库。

第 14 章 需求与方案

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

一句话概括：需求大多来自"老板拍脑袋、产品抄竞品、用户骂娘"；开发的两件保命事——评审会上问清异常流程，估时间永远留 buffer。

新人常见幻觉 vs 现实

幻觉	现实
需求文档会很详细吧？	很多公司 PRD 就是几张截图加几句话
写好代码就能上线吧？	中间还有联调、测试、改 bug、再测试，循环 N 次
测试会把 bug 都发现吧？	线上 bug 永远存在，只是看影响大不大
上线就是点一下按钮吧？	可能点完按钮全公司熬夜救火

评审会：两个核心任务

- 估时间：**产品说"很简单，就加个按钮"，你得脑补背后要改几张表、几个接口、多少页面；
- 挑毛病：**PRD 没写异常流程？问。没写兼容方案？问。没写数据统计？问。**评审会上不问，以后就是你加班补。**

估时间的潜规则：

- 产品给的 deadline 不能当真，按"正常干完 + 联调 + 改 bug"报；
- 你说 3 天，产品和项目经理记成 2 天——报 5 天留 buffer；
- 不当面说"做不了"，要说"可以做，但需要 X 天 / 砍掉 Y 功能"。

需求变更应对

变更类型	应对
小改动（文案、样式）	口头答应顺手做，但在 群里留痕 ，防扯皮
大改动（逻辑、功能）	让产品重新发 PRD/变更单，重新评估工期："能做，但上线时间延后"
临上线改需求	除非老板亲自下令，否则一律拒绝——这叫 封版 ，是保命底线

血泪教训：有同事上线前夜答应改"小逻辑"，牵一发动全身，回滚 3 次通宵到早上 6 点。从此团队规定：**封版后只修 P0 bug，其余一律下版本。**

最小化技术方案（写不写？）

- 大厂必须写方案文档、架构师点头才开工；小厂"先写吧，边写边想"是常态——但你不写，后面联调、交接、线上出问题时就得背锅；
- 最少落到文档/备注这四点：① 改哪些接口/页面，影响范围？② 数据库加字段/表？要不要迁移？③ 依赖哪些第三方？挂了怎么办？④ 并发/性能/安全风险怎么处理？

反面教材："积分抵扣"没考虑并发，两人同时下单积分被扣两次。根因：没写方案，没意识到扣积分不是原子操作。

最后敲个黑板：需求阶段的核心不是"接活"，是"问清楚 + 留 buffer + 留痕"。

第 15 章 开发、联调与 Code Review

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

浓缩成一句：编码期四件事——别裸奔、日志记清、异常别吞、密钥别硬编码；Git 用不好轻则丢代码重则搞崩主分支；联调先对文档，CR 先自己 diff 一遍。

编码期间注意点

- **别裸奔：**至少自测 happy path（主流程能跑通），别写完就扔给测试，会被鄙视；
- **日志记清楚：**关键节点打日志；但别到处 print，线上日志太多会被运维骂；
- **异常别吞掉：**try-catch 至少打 error 日志，别直接 pass——否则线上出事时日志干干净净，你根本不知道怎么死的；
- **敏感信息别硬编码：**密码、密钥、token 放环境变量，代码会被所有人看到。

Git 使用规范（保命用）

- 每天下班前 push，哪怕没写完——硬盘挂了哭都来不及；
- 提交信息写人话：fix：修复优惠券重复领取时库存扣减异常 "fix bug"；
- 合并前先 pull 最新主分支，解决完冲突再提交；
- **永远不要在主分支上开发；永远不要 force push。**

```
feat: 新增XX功能
fix: 修复XX问题
refactor: 重构XX逻辑（没有改功能）
chore: 配置、构建相关改动
```

联调：最痛苦的环节

为什么痛苦：字段名不一致（userId vs user_id）、类型对不上（整数 vs 字符串）、接口没好（"明天好"→"下午好"→"晚上好"）、环境不通。

减少痛苦四招：① 先对文档（Swagger/Apifox/YApi 落全参数与错误码）；② 后端没写好前端先 Mock，别傻等；③ 提前定规范（驼峰 vs 下划线、空值传 null 还是不传）；④ 环境挂了别硬扛，一起找运维。

联调暗语翻译：

- "接口好了" = 主干逻辑通了，边界情况没测；
- "你先看下" = 可能有 bug，但我还没自测；
- "这个数据不对吧" = 大概率是你传错了，但你不好直说。

Code Review：在审什么、怎么应对

审什么	要点
逻辑对不对	空指针、边界条件、并发安全
命名清不清楚	a、b、c 这种变量名直接打回
有没有冗余	同样逻辑写三遍就抽公共方法
安全隐私	SQL 注入、输入校验、敏感信息脱敏

应对： 明显对的乖乖改；觉得不合理的可以讨论但语气要好（"这里我这样做是因为 XXX，你觉得呢？"比"你不懂"强一万倍）；一次被提 50 条别崩溃，分优先级改。

CR 避坑： 提交前自己先 diff 一遍；描述写清"改了什么、为什么改、怎么验证的"；别在 CR 里做需求变更。

小结一下： 代码是写给人看也写给 AI 看的；日志、Git、文档是你的"护身符"，别省。

第 16 章 测试与上线

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

掏心窝说一句：QA 是你这期间的"敌人"，也是帮你挡子弹的人；上线前必过 checklist、必想好回滚方案；上线后至少观察 30-60 分钟。

测试四板斧

类型	测什么
功能测试	对着 PRD 一条一条过
边界测试	空输入/超长/并发点击——最容易出 bug
兼容性测试	不同浏览器/机型/系统——前端最头疼
回归测试	改一处测周边，防止改 A 坏 B

Bug 生命周期：测试发现 → 提 bug 单（复现步骤、截图、期望、实际）→ 开发确认修复 → 状态改"已修复" → 测试复测关闭 / 不通过重开。误报可以拒绝，但要有理有据。

和测试相处的原则：

- 不说"我这边是好的"，要说"我在 XX 环境复测了，确实没问题"；
- 不私下改 bug 不更新单子——测试不知道你改了不会复测；
- 复现不了厚着脸皮当面看，别硬扛；
- 提测前自己先跑一遍主流程；bug 当天响应；修完告知验证入口。

上线前 checklist

- ☐ 代码已合并到发布分支且最新
- ☐ 配置已配好（数据库连接、第三方 key、开关）
- ☐ 数据库变更脚本已在预发布环境跑过
- ☐ **回滚方案明确**
- ☐ 产品/测试已知会，有人值班

三种发布方式

方式	适用	说明
直接发布	小功能、低风险	直接全量，小厂常见
灰度发布	常规功能	先 1%/5%/10% 观察，最稳妥
蓝绿部署	关键服务	新旧两套环境，验证后切流量，出问题秒切回

上线当晚： 盯监控大屏、每 5 分钟刷错误率、产品问"好了吗"你说"再观察 10 分钟"、发现异常**第一时间回滚**，别想着热修复。

上线后： 观察 30-60 分钟核心指标 → 产品/运营验收 → 次日复盘；有问题写事故报告。

浓缩一句： 测试是帮你挡子弹的；上线最贵的不是发布，是"回滚方案"。

第 17 章 救火与复盘

基础版本 • V0.1

难度：入门 | 更新时间：2026-08-14

开门见山：线上问题处理顺序永远是"先止血，再看病"——能回滚就回滚，日志是你的命根子。

故障分级

级别	定义	处理要求
P0（致命）	核心功能不可用/数据丢失/资金安全	立刻回滚/热修，全员 oncall
P1（严重）	主要功能异常，有绕过方案	2 小时内修复或回滚
P2（一般）	非核心功能异常	下个版本修
P3（轻微）	UI 问题、文案错误	有空再说

处理流程五步

- 发现：**监控告警、用户投诉、客服反馈；
- 止损：**能回滚就回滚，不能就降级/关开关，先恢复服务；
- 定位：**查日志、查链路、复现，找根因——**日志是你的命根子**；
- 修复验证：**修完观察一段时间再算结案；
- 复盘：**写事故报告（时间线、根因、改进措施）——不是批斗会，是为了下次不犯。

日志基本功：会按时间段/关键字/用户 ID 搜日志；知道日志存哪（文件/ELK/阿里云 SLS）；学会看链路追踪（SkyWalking、Jaeger）。

各角色到底在干什么

角色	核心工作	你打交道时
产品经理	提需求、定优先级、验收	需求不明确找他对
项目经理	排期、催进度、协调资源	周报日报发给他，延期提前说
测试工程师	写用例、测功能、提 bug	别吵，bug 修完主动告知

角色	核心工作	你打交道时
前端/后端	写页面/写接口	接口文档给清楚，字段别偷改
运维/SRE	部署、监控、报警	上线前要环境权限，告警别静音
设计师	UI 稿、切图、走查	实现差距大提前沟通

小公司一人身兼数职，大厂分工细但沟通成本高，各有好坏。

给新人的生存建议

- 日报/周报及时写，哪怕只写"开发 XX 功能，联调中"——领导看不到进度会慌；
- 开会做笔记、会后发纪要——口说无凭，记录是甩锅神器；
- 不懂就问，但先自己搜；同样的问题别问三遍；
- 接任务先复述："我理解是要做 XX，对吗？"
- 别在群里 @老板问技术细节；别封版后偷偷改代码；别说"应该没问题"——那是线上故障的预言，要说"我验证过了，在 XX 环境正常"。

带走这一句：流程是死的，人是活的——保持沟通、留好记录、该甩锅时别怂，该背锅时别躲。

第 18 章 智能体开发六步法（企业级）

基础版本 · V0.1

难度：进阶 | 更新时间：2026-08-14

核心就在这：智能体不是"写个 Prompt 调个接口"就完事——它必须靠"评测 + 日志 + 监控"三位一体约束；上线前：评测集通过率 $\geq 90\%$ 、核心场景零阻塞、安全过滤生效。

六步标准流程

步骤	做什么	关键产出
1 需求澄清	明确目标（客服/生成/分析/自动化）；交互方式（同步/异步）；"必答清单"和"必须拒绝的场景清单"；合规确认	场景定义文档
2 技术选型	选基座模型（GPT-5.6/Claude/DeepSeek/GLM-5/Kimi K3）；系统提示词 + Git 版本管理；工具 schema；记忆策略（N 轮/摘要/向量）；异常降级 + 内容安全层	架构设计
3 评测集构建	收集 100+ 条真实 query（高频/低频/边界）；标注理想回答/必含信息点/禁止内容；自动化评测脚本（规则或 LLM-as-a-Judge）； 每次改 Prompt 必须全量回归	评测集
4 开发实现	Prompt/工具/记忆/后处理模块化；全链路日志（输入/上下文/原始输出/耗时）；SSE 流式 + "停止生成"按钮；QPS 限流与熔断	可维护代码
5 联调验收	与产品对齐语气/格式/统一"不知道"话术；测试集跑分写入验收报告（通过率 + 阻塞项）；配置调用成功率/延迟/Token 告警阈值	验收报告
6 上线迭代	灰度 1%/5% $\rightarrow$ 全量；收集点赞/点踩/纠错回流优化；A/B 实验决定 Prompt/模型/Temperature	持续优化

骨子里的道理：传统代码靠测试兜底，智能体靠"评测集 + 日志 + 监控"三件套兜底——不确定性越高，流程越要死板。

第 19 章 标准工作流与提示词模板库

基础版本 · V0.1

难度：进阶 | 更新时间：2026-08-14

先说结论：标准工作流 = 需求拆解 → 计划 → 小步实现 → 测试 → 审查 → 提交复盘；好 Prompt 有四个要素——目标、上下文、输出格式、边界条件；本模板库可直接复制使用。

为什么需要标准工作流

直接一句"帮我做一个网站"不是不行，但 AI 改得很快，你不知道它改了什么、能不能交付。稳定方式：**先计划，后执行；一次只改一小块；每一步都可验证。**

Prompt 四要素（OpenAI 官方 2026 指南）

要素	回答的问题	示例
Goal（目标）	应该做什么？	"把会议记录整理成项目进展通报"
Context（上下文）	哪些信息有帮助？各资料干什么用？	"按品牌手册语气写；旧销售报告只用于了解客户，不引用过期数字"
Output（输出）	什么格式、长度、详细度？	"一页摘要，决策和下一步放最前"
Boundaries（边界）	哪些不能动？哪些要先确认？	"不要改金额和日期；只生成草稿不要发送"

边界条件别写太多——选一两条"最容易造成实际损失"的写死即可。

标准六步法 + 每步模板

第 1 步：需求拆解（先别改代码）

请先帮我做需求拆解，不要立刻修改代码。

需求：【这里写你的需求】

请按下面结构分析：

- 背景是什么（项目阶段 / 为什么要做 / 属于新功能、Bug 修复、UI 优化还是重构）
- 要解决什么问题（具体到什么程度 / 哪些不在本次范围）

3. 哪些文件可能相关（先列出来，不要直接修改）
 4. 哪些功能不能动（不要改哪些逻辑、接口、页面、组件、依赖）
 5. 什么结果算完成（功能/页面/代码的完成标准）
 6. 需要哪些测试（运行什么命令 / 手动检查哪些页面 / 重点验证哪些流程）
 7. 有哪些风险（影响范围 / 样式污染 / 重构过度 / 新增依赖）
- 最后给我一个简短执行建议：先做哪一步，是否需要我确认后再修改。

第 2 步：制定计划

先不要写代码，也不要修改任何文件。
请根据当前需求和项目结构，制定一个修改计划。等我确认后再开始执行。

（Codex 里可用 /plan；Claude Code 里切换 Plan Mode。）

第 3 步：小步实现（一次只改一个功能点）

请开始小步实现。当前只执行第【1】步：【这里写本次只做的一个功能点】
要求：

1. 一次只改这个功能点
2. 只修改和当前功能直接相关的文件
3. 不要顺手重构无关代码、不要改目录结构
4. 不要新增不必要依赖、不要删除已有功能
5. 不要改计划外的文件

修改完成后停止，并输出：

1. 本次修改了哪些文件
 2. 每个文件改了什么
 3. 为什么必要
 4. 有没有改到计划外内容
 5. 有没有潜在风险
 6. 下一步建议
- 如果遇到不确定的地方，请先停下来问我，不要自行决定。

第 4 步：测试（用结果证明，不是听它说"完成"）

请对本次修改进行测试，不要继续新增功能。

请按顺序执行或说明：

1. 单元测试（有则运行，失败说明原因）
2. 类型检查（有无 typecheck 命令）
3. lint（区分本次新增问题和项目原有问题）
4. 构建（失败请说明报错和影响范围）
5. 手动测试（列出页面、操作步骤、每一步预期结果）
6. 浏览器测试（显示 / 控制台报错 / 移动端 / 关键交互）
7. 回归测试（本次修改是否影响旧功能，列出可能受影响的页面组件）

最后输出：哪些通过 / 哪些失败 / 失败原因 / 能否进入下一步。

第 5 步：代码审查（两轮：AI 自审 + 人工审查）

请对本次修改做代码审查，不要继续写代码。

按下面结构审查：

1. 自审：是否只改了计划内文件 / 有无无关重构 / 有无新增依赖 / 有无误删旧逻辑
2. 边界条件：空数据 / 接口失败 / 未登录 / 权限不足 / 重复点击 / 移动端
3. 安全：是否暴露密钥 token / 权限判断缺失 / 输入未校验 / 接口鉴权
4. 删除内容：删了什么、为什么、有没有别处依赖
5. 业务逻辑：是否符合需求、异常路径、是否影响旧规则

最后给出结论：可以继续 / 需要小修 / 需要回退部分 / 需要重新计划。

只审查，不要修改代码；发现问题先说明和建议，等我确认后再改。

涉及登录、支付、权限、数据库的重要改动，建议用**第二个模型交叉审查**（一个写、一个挑错）；但它帮你发现问题，不替你拍板。

第6步：提交与复盘

请整理本次任务的交付记录：

1. 生成 commit message (feat:/fix:/style:/refactor:/docs: + 中文说明)
2. 生成 PR 描述 (本次修改 / 测试结果 / 风险说明 / 需要重点 review 的地方)
3. 沉淀经验：本次踩了什么坑、有什么可复用的规则，建议是否加入 AGENTS.md / CLAUDE.md
4. 判断是否需要更新 README、.env.example、CHANGELOG

最后给出交付结论：是否可以提交、是否还有未完成事项、是否有人工确认风险。

高频任务模板库（直接复制）

读项目（新项目第一次必做）：

请先不要修改任何代码。请阅读当前项目，输出项目理解报告：

1. 技术栈
2. 目录结构（哪些是核心目录、哪些不建议改）
3. 启动方式（依赖安装、启动命令、环境变量）
4. 测试命令（test/lint/typecheck/build，没有就明说）
5. 核心模块
6. 后续修改风险（全局样式、鉴权、接口、数据库等高危区域）

只输出报告，不要改代码，等我确认。

修 Bug：

我遇到一个 bug：

【现象】【复现步骤】【期望结果】【实际结果】【相关文件/页面】

请先定位原因，不要直接修改。先给出：可能原因 / 需要查看的文件 / 修复方案 / 风

险点。
等我确认后再改代码。

加功能：

我想新增一个功能：【功能描述】【入口位置】【交互流程】【视觉要求】【数据来源】【验收标准】
请先阅读相关代码，给出实现计划。不要改无关文件。确认后实现，运行测试并总结 diff。

写测试：

请为以下功能补充测试：【功能描述】【相关文件】【边界情况】
要求：不改业务逻辑；覆盖正常路径、异常路径、边界条件；遵循项目已有测试约定；运行测试并报告结果。

重构：

请重构以下模块：【模块路径】
目标：提高可读性、减少重复代码；保持现有行为不变；不改变公共 API；不引入新依赖。
请先写重构计划，并说明如何验证行为一致。

写文档：

请根据当前项目生成文档：项目简介 / 安装方式 / 启动方式 / 环境变量说明 / 常用命令 / 目录结构 / 开发注意事项 / 常见问题。
不要编造不存在的命令，必须基于项目文件判断。

2026 省钱 workflow（Codex 并入 ChatGPT 后）

场景	正确姿势
想方案、聊需求	用 Chat（走聊天额度，不烧 Agent 额度）
做成品（PPT/报告/调研）	用 Work，先 Plan 再执行
动项目文件	用 Codex，默认 Terra、简单 Luna、关键才 Sol
大任务	先 Plan 再执行；一个任务只保留相关上下文（善用缓存省 90%）

记住这点就够了：把 workflow 写成模板、把模板用成习惯——AI 负责执行，你负责拍板。

适用对象：无技术背景的普通人、上班族、宝妈、学生、自由职业者。核心提醒：**AI 不会让你自动赚钱，但它能让"你一个人"干出"原来需要一个团队"的产出。**

第 20 章 变现核心逻辑

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

大白话讲：AI 变现三种层级——卖时间（接单）、卖成果（产品）、卖系统（资产）；判断值不值得做的标准只有一个：你的时薪。

为什么 2026 年仍是普通人最好的入场时机

- 工具零代码化：**Coze（扣子）、Dify、n8n 让"会说话就能搭 Agent"成为现实；
- 企业付费意愿觉醒：**中小企业老板知道 AI 能省钱，但他们不会用——这正是你的机会；
- 协议普及：**MCP + A2A 双协议让 Agent 能跨软件、跨智能体协作，功能边界大幅扩展；
- 政策红利：**《数字经济促进法》实施细则明确支持低门槛数字服务岗位。

三种变现层级

层级	本质	月收入预期
第一层：接单型（卖时间）	用 AI 提高效率，接更多单	1,000–10,000 元
第二层：产品型（卖成果）	把 AI 能力打包成标准化服务/产品	5,000–30,000 元
第三层：资产型（卖系统）	搭建自动运行的 AI 系统，睡后收入	10,000–100,000+ 元

普通人从第一层起步，跑通一个行业后向第二、三层升级。

通用算账公式

你的时薪 = 客单价 ÷ 你实际投入的时间
AI 的价值 = （原人工耗时 - 现 AI 耗时）× 你的时薪
接单标准：时薪 ≥ 50 元/小时（兼职）或 ≥ 100 元/小时（全职）

举例：公众号推文人工 4 小时，AI 初稿 + 人工润色 1 小时，省 3 小时。收费 200 元/篇、投入 1 小时 → 时薪 200 元，值得做。

说白了就是：AI 不是印钞机，是杠杆——放大你的时间、判断力和交付质量。

第 21 章 14 条变现路径

基础版本 • V0.1

难度：入门 | 更新时间：2026-08-14

本质就一句：写作/设计/自媒体门槛最低，数字人代运营是 2026 最火方向，自动化工作流代搭建是黄金赛道——所有路径的共同心法都是"AI 初稿 + 人工把关"。

速览表

#	路径	门槛	月入预期	热度
1	AI 写作/代写接单	最低	3,000-15,000 元	稳
2	自媒体内容矩阵	低	1,000-20,000 元	稳
3	AI 数字人/短视频代运营	中	6,000-50,000 元	2026 最火
4	本地商家 AI 客服搭建	中	10,000-25,000 元	蓝海
5	AI 电商（闲鱼/跨境）	低	1,000-20,000 元	稳
6	AI 设计（LOGO/头像/壁纸）	低	2,000-10,000 元	稳
7	知识付费/AI 教程	低	1,000-60,000 元	卷
8	AI 自动化服务（工作流代搭建）	中	10,000-100,000 元	黄金赛道
9	AI 教育辅导	中	3,000-30,000 元	稳
10	AI 翻译/本地化	中（需专业）	5,000-15,000 元	稳
11	AI 数据分析/报告	中	5,000-20,000 元	稳
12	AI 招聘/简历服务	低	2,000-30,000 元	季节性强
13	房产 AI 获客助手	中	6,000-15,000 元	垂直蓝海
14	法律/合同 AI 辅助	高（需专业）	5,000-15,000 元	谨慎

各路径要点

1. AI 写作/代写 门槛最低

- 流程：训练专属写作 Agent（存系统提示词）→ AI 初稿 5 分钟 → 人工查事实/调语气 15-30 分钟 → 交付 → 沉淀私域复购；
- 价格：小红书笔记 50-150 元/篇；公众号推文 200-800 元/篇；简历优化 80-200 元/份。固定 5 客户月产 30 篇 = 12,000 元/月，每天约 3 小时；
- 工具：ChatGPT/Claude/DeepSeek/Kimi；ZeroGPT 查 AI 痕迹、秘塔写作猫润色；渠道：猪八戒、BOSS（搜"远程文案"）、闲鱼、小红书；国外 Upwork、Fiverr（\$30-100/篇）。

2. 自媒体内容矩阵

- 选垂直领域 → AI 每天 3-5 篇（热点+观点+场景图）→ 多平台分发 → 过门槛接广告（小红书 1,000 粉、公众号 500 粉开流量主）；
- 收入：头条分成 1,000-5,000 元/月；公众号流量主 5,000-20,000 元；小红书 1,000-10,000 元（低粉也能高变现）；
- 关键认知：**粉丝量不是唯一指标，核心是"垂直 + 转化"**。

3. AI 数字人/短视频代运营 2026 最火（非直播）

- 帮本地商家做"不用真人出镜"的产品视频、口播、直播切片：HeyGen/D-ID 生成数字人 → ChatGPT 写脚本 → 剪映剪辑 → 交付或代发布；
- 价格：单条 100-500 元；包月 2,000-5,000 元/客户；直播搭建 3,000-10,000 元/次。接 10 客户 × 10 条 = 10,000-50,000 元；
- 2026 趋势：**AI 艺人/数字人广告报价已达 25 万元/条量级**——做代运营的同时可以考虑孵化自己的数字人 IP；
- 案例：外卖员用数字人卖居家清洁神器，每晚 1 小时，月纯利 1.2 万。

4. 本地商家 AI 客服 蓝海

- Coze/Dify 搭客服 Agent → 上传 FAQ → 接入公众号/小程序/抖音企业号 → 调优到能答 80% 常见问题 → 收搭建费 + 月维护费；
- 价格：搭建费 1,000-5,000 元/家；月维护 300-1,000 元。月新签 5 家 + 存量 20 家 ≈ 20,000 元/月；
- 案例：专攻"牙科诊所 AI 客服"，同一套模板改知识库卖不同诊所，半年签 30 家，月入 25,000 元；
- 别报天价，细水长流。

5. AI 电商

- 闲鱼：1688 选品 → AI 写标题 → 代发赚差价；小红书：AI 测评图文挂链接赚佣金；跨境：AI 写多语言 Listing + 自动回复；
- 收入：闲鱼单店 500-3,000 元/月，5 店矩阵 3,000-10,000 元；跨境单店 3,000-20,000 元。

6. AI 设计

- LOGO 200-1,000 元/个；情侣头像 50-200 元/对；壁纸包 9.9-29.9 元被动收入；表情包爆款几千到几万；
- 流程：Midjourney/即梦生成初稿 5 分钟 → Canva/稿定微调 10-20 分钟 → 交付；国外 Fiverr（LOGO \$50-300）、Etsy。

7. 知识付费/AI 教程

- 选具体场景（《HR 怎么用 DeepSeek 写招聘 JD，每篇省 40 分钟》）→ 免费发 3-5 篇验证需求 → 99 元电子书/199 元视频课/999 元训练营 → 私域成交；
- 案例：3 篇"AI 做 PPT"笔记涨粉 5,000，99 元课首月卖 400 份 = 39,600 元；
- 群规：不允许纯割韭菜——除非自成一套体系。

8. AI 自动化服务 黄金赛道

- 学 n8n 或 Coze（搭积木式零代码）→ 选高频痛点（"自动抓竞品价格生成日报"）→ 免费 Demo 展示省多少时间（**交付物不能只是 Demo，必须可维护**）→ 收费搭建 + 月维护；
- 价格：单条工作流 1,000-5,000 元；复杂系统 5,000-20,000 元；月维护 500-2,000 元；企业年服务 50,000-200,000 元；
- 案例：n8n 帮淘宝卖家搭"抓竞品价→Excel→邮件提醒"，3,000 元/套月销 10 套 = 30,000 元/月；
- 延伸：帮企业搭"多智能体协同工作流"（MCP + A2A）是更新蓝海，客单价更高。

9. AI 教育辅导

- AI 生成个性化练习题 → AI 批改 → AI 出学习报告 → 1 对多小班（AI 管个性化、你管互动答疑）；
- 收入：1 对 1 80-200 元/小时；小班 50-100 元/人/课；AI 伴学会员 99-299 元/月（100 会员 = 1-3 万）；
- 学科培训注意双减政策，素质教育/成人教育相对宽松。

10. AI 翻译（仅专业人员）

- AI 初译（DeepL/ChatGPT/Claude）→ 人工校对术语/文化适配 → 交付；
- 价格：中译英 80-150 元/千字；专业领域 200-500 元/千字；每天 3,000 字 ≈ 7,920 元/月（6 小时压到 2 小时）。

11. AI 数据分析/报告

- 数据清洗（1 万行）300-800 元；PPT（20 页）500-2,000 元；行业报告 1,000-5,000 元；数据看板 2,000-10,000 元；
- 案例：财务背景自由职业者月度经营分析报告 2,000 元/月 × 8 家 = 16,000 元/月，投入约 20 小时；
- PPT 类已饱和和最好有设计基础；**前端 Skill 较缺（机会点）**。

12. AI 招聘/简历（金三银四、秋招刚需）

- AI 分析 JD 提取关键词重写简历 → AI 生成模拟面试 → 人工辅导 30 分钟；
- 简历优化 80-300 元/份（月 50 份 = 4,000-15,000 元）；求职陪跑 1,000-3,000 元/月；企业招聘代理按入职 3,000-10,000 元/人。

13. 房产 AI 获客助手

- AI 生成房源文案（每房源 10 条角度）→ Coze 搭自动回复接入中介微信 → 对话自动打标签；

- 文案包 200–500 元；AI 客服搭建 2,000–5,000 元；月维护 1,500–3,000 元/月。

14. 法律/合同 AI 辅助（非专业人员不要碰）

- AI 标风险点 → 人工复核补本地法规 → 出易懂报告；合同审查 200–800 元/份；**非执业律师不得提供法律意见，必须标注"仅供参考，不构成法律建议"。**

最后提一句：14 条路挑 1 条你最熟悉的起步；所有路径通用公式：AI 出初稿、你出判断。

第 22 章 起步路线图与避坑

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

核心就一点：五阶段路线——选赛道 → 免费获客攒案例 → 提价转私域 → 矩阵复制 → 资产化；五大陷阱第一条就是"日入过万"神话。

从 0 到 1 路线图

时间	动作
第 1 周	选 1 个最感兴趣、有一点点行业知识的方向；注册工具；花 3 天做出 3 个"样例作品"（3 篇笔记/3 个 LOGO/1 个客服 Demo）
第 2 周	闲鱼/小红书/朋友圈发"免费试做""首单半价"；接 3-5 个低价单， 目的是攒案例和评价，不是赚钱 ；案例截图保存
第 3-4 周	有 3-5 个案例后提到市场均价；引导加微信转私域；建标准化交付流程（SOP）
第 2-3 月	流程复制到同行业更多客户；重复工作进一步 AI 化；服务打包成标准产品
第 4-6 月	收入稳定过万后招兼职助理；工作流模板化卖给同行；升级"资产型"，搭自动运行系统

五大常见陷阱

- "日入过万"神话：**真实 AI 副业时薪 15-50 元起步，熟练后 50-200 元。**任何先交钱的"培训"都要警惕；**
- 纯 AI 复制粘贴：**平台算法和用户都不傻，纯 AI 内容被降权、客户不复购——**必须人工判断和个性化；**
- 忽视版权：**AI 生成内容可能有版权争议，商用选有商用授权的工具；
- 接单没有合同：**小单也要文字明确需求、价格、修改次数、交付时间；
- 不做私域：**平台随时封号改规则，客户必须沉淀到微信/公众号/社群。

法律责任边界

领域	可以做	不可以做
医疗	健康科普（标注仅供参考）	诊断、开药、治疗建议
法律	文档整理、信息辅助（标注不构成法律意见）	出具正式法律意见
金融	数据整理、报告撰写	投资建议
教育	素质教育、成人教育	违规学科培训

零成本启动工具清单

用途	工具
对话/写作	ChatGPT、Claude、DeepSeek、Kimi、豆包（免费版够用）
绘图	Midjourney（付费）、即梦 AI、可灵、Stable Diffusion
视频/数字人	剪映、HeyGen、D-ID、硅基智能
Agent 搭建	Coze（扣子）、Dify、FastGPT、n8n
PPT/文档	Gamma、ChatPPT、WPS AI、讯飞智文
分发	蚁小二、易撰

核心心法：2026 年的 AI 赛道像极了 2016 年的抖音——所有人都知道这是未来，但大多数人没有当回事。**别等完美，先上车。**用你最熟悉的领域搭最小可用产品，扔到市场看反应，数据会告诉你往哪走。

划个重点：学习、从事任何行业都是贵在坚持——风口上也要脚踏实地，半途而废肯定没有结果。

这一篇能让你少走弯路、少亏钱、少踩法律红线。新手请务必读完。

第 23 章 权限与安全铁律

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

说穿了就是：使用智能体的五条铁律——当实习生不当神、权限最小化、先备份、人工验收、敏感信息别喂给它；检查 Skill 恶意代码有五个标准动作。

五条铁律

- 当实习生，不当神：**能解决大多数问题，但不能解决所有问题，需要你引导、限制、验收；
- 权限最小化：**不要开"完全访问"——就像把电脑密码给朋友，他"好心"删了你的文件你还不知道。沙盒模式选"请求批准"，danger-full-access 新手禁用；
- 重要操作先备份：**批量改文件前先列改动清单，或先 Git 提交；
- 关键结果人工验收：**代码、文案、数据上线前自己过一遍；
- 敏感信息别喂给它：**身份证、密码、密钥、客户隐私，默认它"记不住"也不该知道。

检查 Skill / 插件是否含恶意代码（五步）

- 看来源：**只从官方市场、作者官方仓库、高 star 且近期活跃的 GitHub 仓库下载；QQ 群文件、网盘分享要警惕；
- 看内容：**重点找——下载并执行远程代码的命令（curl/wget 管道执行）、读取敏感文件（浏览器 cookie、SSH 密钥、~.ssh）、上传数据的代码、base64 混淆长串；
- 看权限声明：**好的 Skill 会写明"需要什么权限、为什么需要"；不声明就索要权限的要警惕；
- 小范围先试：**先在无敏感数据的测试项目里跑一遍，观察行为；
- 汉化包/修改版警惕：**官方语言包优先（第三方汉化包可能藏恶意代码）。

关于"后门"与破解版工具

- 原教程提醒：**Claude Code 2.1.91-2.1.196 被社区报告存在后门风险**——只从官方渠道安装、保持最新版本（npm update -g @anthropic-ai/claude-code）；
- 一切"破解版""魔改版""带梯子版"安装包都有植入风险，别图省事；
- "绕封锁安装""接码注册"涉及账号服务条款和所在地法律法规，**本教程仅作信息说明，不推荐、不教学**，请使用官方支持的正规渠道。

网络与中转风险

- 部分海外工具在部分地区不可直接访问，是否使用、如何访问请自行判断并遵守所在地法律法规；

- **不建议用中转站：**虚标、降智、跑路是常态；若必须用，选有信誉、标明上游的服务，且**不要用中转站处理敏感数据；**
- API Key 像银行卡密码一样保管：泄露可能被刷爆账单，用量异常立即检查/重置。

本地模型的安全优势

数据不出本机，适合隐私敏感内容；但本地模型能力弱于云端旗舰——别用 7B 模型做复杂推理然后骂它笨；模型只从官方库（ollama.com/library）拉取。

危险命令红线（看到先喊停）

命令/操作	风险
rm -rf、del /s、rd /s、Remove-Item -Recurse	批量删除文件/目录
git reset --hard、git clean -fd	丢失未提交改动
curl xxx sh / iex 远程脚本	下载并直接执行未知代码
--yolo、danger-full-access、跳过审批参数	放开全部权限
sudo	修改系统级内容

应对：先让它解释"这条命令的作用、风险、有没有更安全的替代方案"；全局 AGENTS.md 里可以直接写死"禁止批量删除，只能一次删一个明确路径的文件"。

带走这句话：把智能体关进围栏（沙盒+审批），把 Skill 当陌生人审，把 Key 当密码管。

第 24 章 法律红线与合规速查

基础版本 · V0.1

难度：入门 | 更新时间：2026-08-14

一句话讲明白：医疗、法律、金融、教育四个领域都有硬红线；变现路上"仅供参考"四个字是你的免责护身符。

领域	可以做	不可以做
医疗	健康科普（标注仅供参考）	诊断、开药、治疗建议
法律	文档整理、信息辅助、模板（标注不构成法律意见）	出具正式法律意见、代理诉讼
金融	数据整理、报告撰写	投资建议、荐股
教育	素质教育、成人教育	违规学科培训（关注双减政策）
内容	AI 辅助创作 + 人工把关	纯 AI 批量灌水、侵权搬运
数字人	商业宣传（注明 AI 生成）、自有 IP	未经授权使用真人肖像/声音
数据	脱敏后的分析	未经同意处理客户隐私、上传敏感数据

收个尾：能力越大边界越清——AI 帮你干活，法律责任永远在你。

附录 A：命令速查总表

基础版本 • V0.1

Claude Code

```
npm install -g @anthropic-ai/claude-code # 安装
claude / claude -c # 启动 / 恢复上次会话
/compact /clear /memory /init /hooks /rewind /tasks
Ctrl+B 后台 Ctrl+T 任务面板 Alt+V 粘贴图片 Shift+Tab 切模式
```

Codex

```
npm install -g @openai/codex # 安装 CLI
codex / codex login / codex doctor # 启动 / 登录 / 诊断
codex exec "任务" # 一次性执行
codex resume --last / codex archive # 恢复 / 归档
codex mcp add 名称 -- 启动命令 # 添加 MCP
/diff /plan /model /permissions /init /compact /review /quit
```

Hermes

```
irm https://hermes-agent.nousresearch.com/install.ps1 | iex # 安装
hermes model # 配置模型
hermes skills browse # 浏览 Skill 市场
```

Ollama

```
ollama run llama3 / ollama pull mistral / ollama list / ollama rm llama3
ollama show --modelfile llama3 / ollama ps / ollama serve
# 本地接入: http://localhost:11434
```

国产工具

```
npm install -g @deveco/deveco-code # 华为 Deveco Code
```

```
deveco # 启动
```

环境验证

```
git --version # Git 安装验证  
node -v && npm -v # Node 安装验证  
python --version # Python 安装验证
```

附录 B：工具与资源链接汇总（2026-08 更新）

基础版本 · V0.1

API / 模型平台

名称	地址
DeepSeek	https://platform.deepseek.com/
豆包（火山方舟）	https://www.volcengine.com/product/ark
小米 MiMo	https://platform.xiaomimimo.com/console/balance
智谱 GLM	https://open.bigmodel.cn/
月之暗面 Kimi	https://platform.moonshot.cn/
NVIDIA Build（免费额度）	https://build.nvidia.com/settings/api-keys
Agnes AI（免费额度）	https://agnes-ai.com/

智能体工具

名称	地址
Claude Code	https://code.claude.com/
Codex	https://chatgpt.com/codex
Hermes	https://hermes-agent.nousresearch.com/
ZCode（智谱）	https://zcode.z.ai/cn
CodeArts（华为云）	https://codearts.huaweicloud.com/
CC Switch	https://github.com/farion1231/cc-switch/releases
Codex++（第三方接入，谨慎）	https://github.com/BigPizzaV3/CodexPlusPlus/releases
VS Code	https://code.visualstudio.com/
Ollama	https://ollama.com/

环境依赖

名称	地址
Git	https://git-scm.com/download/win
Node.js	https://nodejs.org/zh-cn/download/
Python	https://www.python.org/downloads/windows/

Skill 生态

名称	地址
Agent Skills 规范	https://agentskills.io/specification
Hermes 命令文档	https://hermesagent.org.cn/docs/reference/cli-commands
Superpowers / baoyu-skills / Agent-Reach / find-skills	见第 10 章表格

变现工具

类别	工具
Agent 搭建	Coze (coze.cn)、Dify、FastGPT、n8n、Make、Zapier
数字人	HeyGen、D-ID、硅基智能、小冰；配音 ElevenLabs、魔音工坊
设计	Canva (canva.cn)、稿定设计、创客贴、Uiverse.io (前端动效组件库)
视频	剪映、CapCut；白嫖视频制作 chatopens.com/Web/List.html
分发	蚁小二、易撰

AI 漫剧及视频开源工具（原教程收录）：

- 1. Pixelle-Video（AI 全自动短视频引擎）：<https://github.com/AIDC-AI/Pixelle-Video>
- 2. Waoowao：<https://github.com/waoowao/waoowao>
- 3. Huobao-drama：<https://github.com/chatfire-AI/huobao-drama>
- 4. Toonflow：<https://github.com/HBAI-Ltd/Toonflow-app>
- 5. ComfyUI（本地部署，中高配置）：<https://comfy.org/zh-CN/>

使用 Skill 时最好跟上 Skill 名称，不然相似的 Skill 会"打架"。随时可以让智能体列一份 Skill 介绍清单。

公益推荐

- 帮助走失人员回家：<https://www.xunqinw.cn>

附录 C：常见报错速查表

基础版本 · V0.1

安装和使用智能体时最常见的报错，按 Ctrl+F 搜关键词。

报错关键词	原因	解决方法
command not found / 不是内部命令	软件没装好或 PATH 没配	重装该软件，装完 关掉旧终端、开新窗口 再试
禁止运行脚本 / Execution Policy	Windows PowerShell 默认禁止脚本	管理员开 PowerShell，输入 Set-ExecutionPolicy RemoteSigned，输 Y
npm ERR / EACCES / permission denied	权限不够	管理员身份开终端再装；或用 npm config set prefix 改到用户目录
Authentication failed / 401	API KEY 错了或过期	回平台重新复制 Key，检查多余空格；确认 Key 没写在引号里
Rate limit / 429	调用太频繁，触发限速	等 1 分钟再试；免费额度用完需充值或换平台
Connection refused / ECONNREFUSED	本地服务没启动	检查 Ollama 等本地服务是否在运行（ollama ps）
base_url 错误 / model not found	Base URL 格式错或模型名写错	Base URL 结尾必须是 /v1；模型名去平台文档复制，别手打
token 不足 / context too long	对话太长超出上下文窗口	输入 /compact 压缩对话；或开新会话
Python / pip 不是内部命令	Python 安装时没勾 Add to PATH	重装 Python，勾选 Add Python to PATH ；或手动加 PATH

90% 的安装问题都能解决：把**完整报错信息**复制粘贴给智能体，问它"我是新手，请用大白话解释这个报错并告诉我怎么解决"。

附录 D：各平台费用与模型速查表

基础版本 · V0.1

2026 年 8 月各平台免费额度和大致价格，仅供参考，以官网最新公告为准。

平台	免费额度	大致价格（每百万 token）	推荐场景
DeepSeek	注册送余额	输入 ¥1 / 输出 ¥2	日常代码、写作（性价比最高）
豆包（火山方舟）	新用户送 50 万 token	输入 ¥0.8 / 输出 ¥2	中文写作、对话
智谱 GLM	新用户送额度	输入 ¥1 / 输出 ¥1	代码、通用任务
小米 MiMo	注册送额度	免费期内 ¥0	尝鲜、测试
月之暗面 Kimi	注册送额度	输入 ¥12 / 输出 ¥12	长文档分析（200 万字上下文）
NVIDIA Build	每月 1000 次调用	免费额度内 ¥0	免费体验海外模型
Ollama（本地）	完全免费	¥0（吃硬件电费）	隐私优先、离线使用
Claude（Anthropic）	无免费额度	输入 \$3 / 输出 \$15	代码质量最佳、复杂推理
OpenAI GPT-4o	无免费额度	输入 \$2.5 / 输出 \$10	通用、多模态（看图）

小白算账：写一个网页大概花 2000–5000 token，按 DeepSeek 算大约 ¥0.01–0.02，几乎等于免费。真正费钱的是长文档分析和大量批量任务——先从小任务开始，不用担心费用。

附录 E：小白术语表

基础版本 · V0.1

读到看不懂的词，按 Ctrl+F 搜这张表。

术语	大白话翻译
Agent / 智能体	配齐工具和经验、能自主干活的 AI 实习生
LLM / 大模型	AI 的"脑子"，负责思考和说话的部分
Token	AI 按字数收费的单位（大约 1 个汉字 ≈ 1-2 个 token）
API KEY	绑着账户余额的"门禁卡"，别截图、别发群
Base URL	请求的"收件地址"，告诉智能体把话寄到哪
终端 / CLI / 命令行	打命令的黑窗口（Win+R → 输入 cmd → 回车）
PowerShell	Windows 自带的蓝色终端，比 cmd 更强
npm	Node.js 的"应用商店"，用来安装各种工具
PATH	系统的"通讯录"；软件不登记进去，命令就找不到它
diff	改动的"前后对照表"：绿色 = 新增，红色 = 删除
沙盒	给智能体围的"儿童围栏"，围栏外它碰不到
请求批准	围栏模式：智能体想干越界的事，先问你
Skill	写给 AI 的"标准菜谱"，可反复使用的说明书
MCP	连接外部工具的"USB 接口"
插件 Plugin	把 Skill 和 MCP 打包成一键安装的"工具箱"
AGENTS.md / CLAUDE.md	放在项目门口的"员工守则"，智能体每次开工先读
Prompt / 提示词	你对 AI 说的话
上下文	AI 当前能看到的全部背景信息
开源	代码公开、免费用、谁都能检查

术语	大白话翻译
量化	给模型"瘦身", 让低配电脑也能跑
显存	显卡的内存, 越大能跑的本地模型越大
多模态	能读字、看图、听声的 AI
幻觉	AI 一本正经地胡说八道——重要内容必须人工核实
PR / 合并请求	把改动提交给人审查的"作业审批单"
Commit	给当前代码"拍快照", 随时可以回滚
环境变量	系统的"密码保险箱", Key 放这里, 别写进代码

结语

基础版本 · V0.1

三句话送给你：

1. **先动手，再优化**——装上第一个智能体，比读完十份教程有用；
2. **把它当实习生教**——写 Skill、定规范、验收成果，你才是负责人；
3. **风口上也要脚踏实地**——AI 是杠杆不是印钞机，能坚持交付的人才能赚到钱。

教程会持续更新，后续会持续补充新的实践内容。

（拜拜了 你呢）